Deployment

> Part of the Conversational Agent curriculum. This document covers trust boundaries between an agent and its tool source, session-id validation as a public input boundary, secrets, streaming responses, rate limiting, runtime model switching behind an edge-gated admin endpoint, and observability.

1. Trust boundaries between the agent and its tool source

The agent's connection to its MCP server is its **own** boundary, separate from that server's own connection to whatever *it* wraps (see MCP Server, Deployment, §2) — decide it independently rather than assuming one decision covers both legs.

- **Co-located on the same host**: reasonable to treat this connection as already network-isolated and skip application-layer auth on it, the same reasoning as the MCP server's own co-location note.
- **Reaching a remotely deployed tool server**: changes the calculus — that leg now needs its own explicit auth and network-access decision, since it's no longer implicitly protected by sharing a machine.

2. Session/thread identifiers are a real input-validation boundary

Whatever mechanism ties an incoming request to a persisted conversation thread (Core Concepts, §6's checkpointer + thread id) is **attacker-reachable input** the moment the agent's endpoint is public. Validate and bound it explicitly:

- Check the identifier's format rather than accepting an arbitrary string.
- Cap how many distinct threads a long-lived, never-evicting in-memory checkpointer will hold — an unbounded store fed by a public endpoint is an unbounded memory-growth vector.

This is exactly the kind of guard that needs a **direct unit test on the validating function itself** (Testing, §7) — an end-to-end test can look completely fine while a real gap sits inside the validation logic (for instance, a freshly minted identifier silently skipping a registration step it was supposed to go through, invisible from outside because it still happens to work the first time it's used).

3. Secrets and configuration

Model API keys and any other credentials belong in environment or config, never in source — the same configuration-over-hardcoding principle that applies everywhere else. For a single-user or otherwise low-trust-model host, a plain environment variable is a pragmatic, honest default — **as long as you're explicit about what it doesn't protect against**: any process already running as the same local user can typically still read another process's environment (via standard process-inspection tools), even if the configuration file the value originally came from is itself unreadable to anyone else. Know the stronger alternative for when a deployment's actual threat

model calls for it: store the secret in an OS-level keychain or secret store, and fetch it at process start rather than holding the raw value in a static config file at rest at all.

4. Never leak raw exception text to a public caller

An unauthenticated, public-facing agent endpoint should **log the real exception server-side** (for debugging) and return a **fixed, generic error message** to the caller — never the exception's own text. Raw exception text can incidentally reveal internal details (an internal service's address, a stack trace, a config file path) that are nobody's business on the other side of a public API, even when the exception itself wasn't security-sensitive in any way its author intended.

5. Streaming responses to a UI

A multi-step tool-calling loop benefits from a streaming response protocol — Server-Sent Events is a common, simple choice for one-way server→client progress and result streaming — rather than one long blocking request that leaves a caller staring at a single opaque spinner for the entire loop's duration. Let the progress events from Tool Calling and Guardrails, §5 flow over that same channel alongside the eventual final result, so a frontend can render genuine incremental progress instead of a single all-or-nothing wait state.

6. Rate limiting: app layer vs. edge layer

Decide **once, deliberately**, whether rate limiting belongs in the agent's own application code or in whatever already sits in front of it (a reverse proxy, a CDN/edge layer, an API gateway). Duplicating it in both places is wasted effort; skipping it entirely on a public LLM-backed endpoint is a genuine cost-control gap, since every request can trigger one or more paid model calls. If an edge layer already rate-limits the path this service is deployed under, inheriting that coverage is usually sufficient — add application-layer limiting only when that existing coverage demonstrably doesn't extend to this specific endpoint.

7. Independent versioning and deployability

Same reasoning as MCP Server, Deployment, §4 — give the agent its own dependency manifest, deployed and versioned independently of both the tool server it calls and whatever UI surfaces it, even when everything happens to run on the same host for simplicity today.

8. Switching the model at runtime: an admin capability, done safely

Once an agent is live, "which model answers?" becomes an operational decision (cost, speed, a local model for experiments, a rollback when a provider misbehaves). Doing it by editing a service-manager config and restarting works, but it needs shell access and drops live conversations. A small **admin endpoint** is the better shape — provided it is built with these constraints:

- **Allow-list, not free text.** Offer only the provider/model combinations you have actually validated against *this graph's* tool-calling path. A free-text model field lets an admin select something that silently never produces a tool call, with no diagnostic anywhere in the UI. Validate the requested id against the list in the API layer and answer 422 on a miss. Record why excluded models were excluded (tool calls never populate; list arguments serialize as strings) so nobody re-adds them.
- **A clear precedence order for the active choice:** persisted admin choice (if present and still on the allow-list) → environment variables → code default. A malformed or no-longer- allowed stored value is treated as absent — logged, never raised. Startup must never fail because of a stale settings file.
- **Persist outside the repo checkout, atomically.** Write to a temp file and `os.replace` it into place, so a crash mid-write cannot leave a half-written file for the next boot, and make the path overridable so tests never touch a real machine path.
- **Apply by rebuild-and-swap, not restart — and carry the same checkpointer forward.** Build the new graph with the new model *but the same checkpointer instance and the same cached tools*. A rebuild that defaults to a fresh checkpointer drops every live conversation's memory with no error anywhere. Serialize concurrent admin writes with a lock.
- **Order the steps build → persist → swap.** If the swap happened first and the persist then failed (disk full, permissions), the process would run one model while the store names another, and the next restart would silently flip it. On any failure, leave the running graph untouched and return a curated error naming the model that is still serving. In-flight requests need no special handling if each request resolves the current graph at request time.
- **Keep the model-construction function environment-only.** The admin path calls it with explicit overrides; the function itself never reads the settings store. That keeps every graph node's "the LLM is injected" test contract unchanged.
- **Gate it at the edge, with a login policy.** An admin page is for a human, so use an identity-provider **login** policy (a single allow-listed account) rather than the service-token policy used for machine clients. Both the page and the admin API path need their own rule: a login policy answers an unauthenticated request with a redirect to the identity provider, which only a real top-level page navigation can follow — a bare `fetch()` cannot. The page navigation performs the login and sets the cookie; later same-origin `fetch()` calls carry it. No application-layer auth code is needed, and the convention "each service owns its own admin routes — no shared admin service" means a future capability elsewhere adds one more path rule to the same edge application, not a new auth system. (A browser page's own `fetch()` to a gated endpoint may also need the identity provider's domain in the page's content-security `connect-src`, or the redirect fails silently.)

9. Observability: trace each query, time each step

A multi-node graph with a tool loop is opaque from outside. Generate a **trace id per query** and log it with every node and tool call, with durations: then "this answer was slow" becomes "the third tool call took 40 seconds," and "which model was live when it went wrong" is a log search. Bound each model call with an explicit **timeout** — essential for local or self-hosted models, whose latency has a long tail — and treat a timeout as a handled outcome (a curated error to the user), not a hung request. Log tool-call failures and flag a turn whose every tool call failed, so a silently empty answer is distinguishable from a legitimately empty one.

10. Deployment checklist

Concern	Decide
Agent → tool-server auth	Independently of the tool server's own upstream auth; co-location can justify skipping it, a remote server can't
Thread/session ids	Validate format, cap store growth, unit-test the guard directly
Secrets	Env var as a pragmatic default; know the keychain-backed alternative and when it's warranted
Errors to a public caller	Log real detail server-side; return a fixed generic message
Long-running turns	Stream progress + result over one channel (e.g. SSE), not one blocking call
Rate limiting	Pick one layer (app or edge) deliberately; confirm coverage actually reaches this endpoint
Deployability	Own manifest, independently versioned from both its tool source and its UI
Runtime model switch	Allow-list of validated models; persisted outside the repo; build → persist → swap; same checkpointer; edge login policy on both page and API
Observability	Trace id per query; per-node/tool timings; per-call model timeout
Pre-deploy gate	Run the golden-prompt evaluation on the model you ship after any prompt or guardrail change

See also

- Conversational Agent curriculum index
- MCP Server, Deployment for the matching decisions on the tool-source side of §1's boundary
- Testing, §7 for how to actually verify the guards this document requires
- Architecture Overview, §11 for how this fits alongside a more conventional API/frontend deployment